

```
1 //#####
2 // FILE:   LABstarter_main.c
3 //
4 // TITLE:   Lab Starter
5 //#####
6
7 // Included Files
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <stdarg.h>
11 #include <string.h>
12 #include <math.h>
13 #include <limits.h>
14 #include "F28x_Project.h"
15 #include "driverlib.h"
16 #include "device.h"
17 #include "F28379dSerial.h"
18 #include "song.h"
19 #include "dsp.h"
20 #include "fpu32/fpu_rfft.h"
21
22 #define PI          3.1415926535897932384626433832795
23 #define TWOPI       6.283185307179586476925286766559
24 #define HALFPI      1.5707963267948966192313216916398
25 // The Launchpad's CPU Frequency set to 200 you should not change this value
26 #define LAUNCHPAD_CPU_FREQUENCY 200
27
28
29 // Interrupt Service Routines predefinition
30 __interrupt void cpu_timer0_isr(void);
31 __interrupt void cpu_timer1_isr(void);
32 __interrupt void cpu_timer2_isr(void);
33 __interrupt void SWI_isr(void);
34
35 // sjew: fn defs
36 void setEPWM1A(float controleffort);
37 void setEPWM2A(float controleffort);
38
39 void init_eQEPs(void);
40 float readEncLeft(void);
41 float readEncRight(void);
42 float readEncWheel(void);
43
44 // Count variables
45 uint32_t numTimer0calls = 0;
46 uint32_t numSWIcalls = 0;
47 extern uint32_t numRXA;
48 uint16_t UARTPrint = 0;
49
50
51 // sjew: global variables to track velocity, position, effort values
52 int pwm_iterator = 1;
53 int motor_counting_up = 1;
54 float ctrl_effort = 0.0;
55 float curr_left_enc = 0;
56 float curr_right_enc = 0;
57 float curr_p_right = 0;
```

```
58 float curr_p_left = 0;
59 float prev_p_right = 0;
60 float prev_p_left = 0;
61 float v_left = 0;
62 float v_right = 0;
63 float curr_knob_enc = 0;
64 float left_rpf = 9.83;
65 float right_rpf = 9.833;
66 float u_left = 0;
67 float u_right = 0;
68
69 // sjew: friction const
70 float visc_pos = 2.3335;
71 float static_pos = 1.8655;
72 float visc_neg = 2.2755;
73 float static_neg = -1.863;
74
75
76
77 void main(void)
78 {
79     // PLL, WatchDog, enable Peripheral Clocks
80     // This example function is found in the F2837xD_SysCtrl.c file.
81     InitSysCtrl();
82
83     InitGpio();
84
85     // Blue LED on LaunchPad
86     GPIO_SetupPinMux(31, GPIO_MUX_CPU1, 0);
87     GPIO_SetupPinOptions(31, GPIO_OUTPUT, GPIO_PUSHPULL);
88     GpioDataRegs.GPASET.bit.GPIO31 = 1;
89
90     // Red LED on LaunchPad
91     GPIO_SetupPinMux(34, GPIO_MUX_CPU1, 0);
92     GPIO_SetupPinOptions(34, GPIO_OUTPUT, GPIO_PUSHPULL);
93     GpioDataRegs.GPBSET.bit.GPIO34 = 1;
94
95     // LED1 and PWM Pin
96     GPIO_SetupPinMux(22, GPIO_MUX_CPU1, 0);
97     GPIO_SetupPinOptions(22, GPIO_OUTPUT, GPIO_PUSHPULL);
98     GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
99
100    // LED2
101    GPIO_SetupPinMux(94, GPIO_MUX_CPU1, 0);
102    GPIO_SetupPinOptions(94, GPIO_OUTPUT, GPIO_PUSHPULL);
103    GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
104
105    // LED3
106    GPIO_SetupPinMux(95, GPIO_MUX_CPU1, 0);
107    GPIO_SetupPinOptions(95, GPIO_OUTPUT, GPIO_PUSHPULL);
108    GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
109
110    // LED4
111    GPIO_SetupPinMux(97, GPIO_MUX_CPU1, 0);
112    GPIO_SetupPinOptions(97, GPIO_OUTPUT, GPIO_PUSHPULL);
113    GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
114
```

```
115 // LED5
116 GPIO_SetupPinMux(111, GPIO_MUX_CPU1, 0);
117 GPIO_SetupPinOptions(111, GPIO_OUTPUT, GPIO_PUSHPULL);
118 GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
119
120 // LS7366#1 CS
121 GPIO_SetupPinMux(130, GPIO_MUX_CPU1, 0);
122 GPIO_SetupPinOptions(130, GPIO_OUTPUT, GPIO_PUSHPULL);
123 GpioDataRegs.GPECLEAR.bit.GPIO130 = 1;
124
125 // LS7366#2 CS
126 GPIO_SetupPinMux(131, GPIO_MUX_CPU1, 0);
127 GPIO_SetupPinOptions(131, GPIO_OUTPUT, GPIO_PUSHPULL);
128 GpioDataRegs.GPECLEAR.bit.GPIO131 = 1;
129
130 // LS7366#3 CS
131 GPIO_SetupPinMux(25, GPIO_MUX_CPU1, 0);
132 GPIO_SetupPinOptions(25, GPIO_OUTPUT, GPIO_PUSHPULL);
133 GpioDataRegs.GPACLEAR.bit.GPIO25 = 1;
134
135 // LS7366#4 CS
136 GPIO_SetupPinMux(26, GPIO_MUX_CPU1, 0);
137 GPIO_SetupPinOptions(26, GPIO_OUTPUT, GPIO_PUSHPULL);
138 GpioDataRegs.GPACLEAR.bit.GPIO26 = 1;
139
140 // WIZNET RST
141 GPIO_SetupPinMux(27, GPIO_MUX_CPU1, 0);
142 GPIO_SetupPinOptions(27, GPIO_OUTPUT, GPIO_PUSHPULL);
143 GpioDataRegs.GPACLEAR.bit.GPIO27 = 1;
144
145 //PushButton 1
146 GPIO_SetupPinMux(157, GPIO_MUX_CPU1, 0);
147 GPIO_SetupPinOptions(157, GPIO_INPUT, GPIO_PULLUP);
148
149 //PushButton 2
150 GPIO_SetupPinMux(158, GPIO_MUX_CPU1, 0);
151 GPIO_SetupPinOptions(158, GPIO_INPUT, GPIO_PULLUP);
152
153 //PushButton 3
154 GPIO_SetupPinMux(159, GPIO_MUX_CPU1, 0);
155 GPIO_SetupPinOptions(159, GPIO_INPUT, GPIO_PULLUP);
156
157 //PushButton 4
158 GPIO_SetupPinMux(160, GPIO_MUX_CPU1, 0);
159 GPIO_SetupPinOptions(160, GPIO_INPUT, GPIO_PULLUP);
160
161 //SPIRAM CS Chip Select
162 GPIO_SetupPinMux(19, GPIO_MUX_CPU1, 0);
163 GPIO_SetupPinOptions(19, GPIO_OUTPUT, GPIO_PUSHPULL);
164 GpioDataRegs.GPASET.bit.GPIO19 = 1;
165
166 //F28027 CS
167 GPIO_SetupPinMux(29, GPIO_MUX_CPU1, 0);
168 GPIO_SetupPinOptions(29, GPIO_OUTPUT, GPIO_PUSHPULL);
169 GpioDataRegs.GPASET.bit.GPIO29 = 1;
170
171 //MPU9250 CS Chip Select
```

```
172     GPIO_SetupPinMux(66, GPIO_MUX_CPU1, 0);
173     GPIO_SetupPinOptions(66, GPIO_OUTPUT, GPIO_PUSHPULL);
174     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
175
176     //WIZNET CS Chip Select
177     GPIO_SetupPinMux(125, GPIO_MUX_CPU1, 0);
178     GPIO_SetupPinOptions(125, GPIO_OUTPUT, GPIO_PUSHPULL);
179     GpioDataRegs.GPDSET.bit.GPIO125 = 1;
180
181     // Clear all interrupts and initialize PIE vector table:
182     // Disable CPU interrupts
183     DINT;
184
185     // Initialize the PIE control registers to their default state.
186     // The default state is all PIE interrupts disabled and flags
187     // are cleared.
188     // This function is found in the F2837xD_PieCtrl.c file.
189     InitPieCtrl();
190
191     // Disable CPU interrupts and clear all CPU interrupt flags:
192     IER = 0x0000;
193     IFR = 0x0000;
194
195     // Initialize the PIE vector table with pointers to the shell Interrupt
196     // Service Routines (ISR).
197     // This will populate the entire table, even if the interrupt
198     // is not used in this example. This is useful for debug purposes.
199     // The shell ISR routines are found in F2837xD_DefaultIsr.c.
200     // This function is found in F2837xD_PieVect.c.
201     InitPieVectTable();
202
203     // Interrupts that are used in this example are re-mapped to
204     // ISR functions found within this project
205     EALLOW; // This is needed to write to EALLOW protected registers
206     PieVectTable.TIMER0_INT = &cpu_timer0_isr;
207     PieVectTable.TIMER1_INT = &cpu_timer1_isr;
208     PieVectTable.TIMER2_INT = &cpu_timer2_isr;
209     PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
210     PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
211     PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
212     PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
213     PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
214     PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
215     PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
216     PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
217
218     PieVectTable.EMIF_ERROR_INT = &SWI_isr;
219     EDIS; // This is needed to disable write to EALLOW protected registers
220
221
222     // Initialize the CpuTimers Device Peripheral. This function can be
223     // found in F2837xD_CpuTimers.c
224     InitCpuTimers();
225
226     // Configure CPU-Timer 0, 1, and 2 to interrupt every given period:
227     // 200MHz CPU Freq, Period (in uSeconds)
228     ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000);
```

```

229     ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 20000);
230     ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 1000);
231
232     // Enable CpuTimer Interrupt bit TIE
233     CpuTimer0Regs.TCR.all = 0x4000;
234     CpuTimer1Regs.TCR.all = 0x4000;
235     CpuTimer2Regs.TCR.all = 0x4000;
236
237     init_serialSCIA(&SerialA,115200);
238     init_serialSCIB(&SerialB,19200);
239 //     init_serialSCIC(&SerialC,115200);
240 //     init_serialSCID(&SerialD,115200);
241     init_eQEPs();
242
243     GPIO_SetupPinMux(22, GPIO_MUX_CPU1, 5);
244     GPIO_SetupPinMux(0, GPIO_MUX_CPU1, 1);
245     GPIO_SetupPinMux(2, GPIO_MUX_CPU1, 1);
246
247     // PWM setup
248     EALLOW;
249     // sjew: PWM Setup for EPWM12A
250     //     PERIOD: 20kHz, Duty Cycle 50%, High on Zero, Low on Comp A
251     GpioCtrlRegs.GPAPUD.bit.GPIO22 = 1; // For EPW12A
252     // TBCTL
253     EPwm12Regs.TBCTL.bit.FREE_SOFT = 0x2; // Free Soft emulation = Free Run
254     EPwm12Regs.TBCTL.bit.CLKDIV = 0; // Clock divide = 1
255     EPwm12Regs.TBCTL.bit.CTRMODE = 0; // Count up Mode
256     EPwm12Regs.TBCTL.bit.PHSEN = 0; // Do not load time time-base counter from the time-base
phase registry
257     // TBCTR
258     EPwm12Regs.TBCTR = 0; // Start Timer at zero
259     // TBPRD
260     // Set period of the PWM to 20kHz
261     // Counter frequency is 50MHz
262     EPwm12Regs.TBPRD = 2500;
263     // CMPA
264     EPwm12Regs.CMPA.bit.CMPA = 1250; // Duty cycle at 50%, TBPRD / 2
265     EPwm12Regs.AQCTLA.bit.CAU = 0x1; // PWM12A output pin is set low when CMPA is reached
266     EPwm12Regs.AQCTLA.bit.ZRO = 0x2; // the pin be set high when the TBCTR register is zero
267     EPwm12Regs.TBPHS.bit.TBPHS = 0;
268
269     // sjew: setup for ePWM1a
270     //     PERIOD: 20KHz, 50% default duty cycle, HIGH on zero and LOW on compare a
271     GpioCtrlRegs.GPAPUD.bit.GPIO0 = 1;
272     EPwm1Regs.TBCTL.bit.FREE_SOFT = 0x2;
273     EPwm1Regs.TBCTL.bit.CLKDIV = 0;
274     EPwm1Regs.TBCTL.bit.CTRMODE = 0;
275     EPwm1Regs.TBCTL.bit.PHSEN = 0;
276     EPwm1Regs.TBCTR = 0;
277     EPwm1Regs.TBPRD = 2500; // 20 KHz (2500/50MHz)
278     EPwm1Regs.CMPA.bit.CMPA = 1250;
279     EPwm1Regs.AQCTLA.bit.CAU = 0x1; // set low
280     EPwm1Regs.AQCTLA.bit.ZRO = 0x2; // set high
281     EPwm1Regs.TBPHS.bit.TBPHS = 0;
282
283     // sjew: setup for ePWM2a
284     //     PERIOD: 20KHz, 50% default duty cycle, HIGH on zero and LOW on compare a

```

```

285     GpioCtrlRegs.GPAPUD.bit.GPIO2 = 1;
286     EPwm2Regs.TBCTL.bit.FREE_SOFT = 0x2;
287     EPwm2Regs.TBCTL.bit.CLKDIV = 0;
288     EPwm2Regs.TBCTL.bit.CTRMODE = 0;
289     EPwm2Regs.TBCTL.bit.PHSEN = 0;
290     EPwm2Regs.TBCTR = 0;
291     EPwm2Regs.TBPRD = 2500; // 20 KHz (2500/50MHz)
292     EPwm2Regs.CMPA.bit.CMPA = 1250;
293     EPwm2Regs.AQCTLA.bit.CAU = 0x1; // set low
294     EPwm2Regs.AQCTLA.bit.ZRO = 0x2; // set high
295     EPwm2Regs.TBPHS.bit.TBPHS = 0;
296     EDIS;
297
298     // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
299     // which is connected to CPU-Timer 1, and CPU int 14, which is connected
300     // to CPU-Timer 2: int 12 is for the SWI.
301     IER |= M_INT1;
302     IER |= M_INT8; // SCIC SCID
303     IER |= M_INT9; // SCIA
304     IER |= M_INT12;
305     IER |= M_INT13;
306     IER |= M_INT14;
307
308     // Enable TINT0 in the PIE: Group 1 interrupt 7
309     PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
310     // Enable SWI in the PIE: Group 12 interrupt 9
311     PieCtrlRegs.PIEIER12.bit.INTx9 = 1;
312
313     // Enable global Interrupts and higher priority real-time debug events
314     EINT; // Enable Global interrupt INTM
315     ERTM; // Enable Global realtime interrupt DBGM
316
317
318     // IDLE loop. Just sit and loop forever (optional):
319     while(1)
320     {
321         if (UARTPrint == 1 ) {
322             // Normally on the Robot Car we only use the below UART_printfLine functions to
323             // write to the
324             // on board LCD screen. This below serial_printf is only used in lab 1 to print
325             // to a serial
326             // terminal over a USB cable like you will for your Homeworks.
327             //serial_printf(&SerialA,"Num Timer2:%ld Num SerialRX: %ld\r\n",CpuTimer2.InterruptCount,numRXA);
328
329             //IMPORTANT!! %ld is for an int32_t. To print an int16_t use %d
330             // UART_printfLine(1,"Timer2 Calls %ld",CpuTimer2.InterruptCount);
331             // UART_printfLine(2,"T0 %ld,T1
332             %ld",CpuTimer0.InterruptCount,CpuTimer1.InterruptCount);
333             // sjew: print out velocity in fps on top row and knob input on bottom
334             UART_printfLine(1, "L: %.2f, R: %.2f", v_left, v_right);
335             UART_printfLine(2, "Knob: %.3f", curr_knob_enc);
336             UARTPrint = 0;
337         }
338     }
339 }
340 }
341 }
342 }
343 }
344 }
345 }
346 }
347 }

```

```
338
339 // SWI_isr, Using this interrupt as a Software started interrupt
340 __interrupt void SWI_isr(void) {
341
342     // These three lines of code allow SWI_isr, to be interrupted by other interrupt functions
343     // making it lower priority than all other Hardware interrupts.
344     PieCtrlRegs.PIEACK.all = PIEACK_GROUP12;
345     asm("        NOP");           // Wait one cycle
346     EINT;                         // Clear INTM to enable interrupts
347
348
349
350     // Insert SWI ISR Code here.....
351
352
353     numSWIcalls++;
354
355     DINT;
356
357 }
358
359 // cpu_timer0_isr - CPU Timer0 ISR
360 __interrupt void cpu_timer0_isr(void)
361 {
362     CpuTimer0.InterruptCount++;
363
364     numTimer0calls++;
365
366     if ((numTimer0calls%50) == 0) {
367         PieCtrlRegs.PIEIFR12.bit.INTx9 = 1; // Manually cause the interrupt for the SWI
368     }
369
370     if ((numTimer0calls%5) == 0) {
371         // Blink LaunchPad Red LED
372         GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
373     }
374
375
376     // Acknowledge this interrupt to receive more interrupts from group 1
377     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
378 }
379
380 // cpu_timer1_isr - CPU Timer1 ISR
381 __interrupt void cpu_timer1_isr(void)
382 {
383
384     CpuTimer1.InterruptCount++;
385 }
386
387 // cpu_timer2_isr CPU Timer2 ISR
388 __interrupt void cpu_timer2_isr(void)
389 {
390     // Blink LaunchPad Blue LED
391     GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
392     CpuTimer2.InterruptCount++;
393
394     // sjew: CODE FOR LAB 2 WEEK 1 oscillating motor
```

```
395 // sjew: if cmpa is equal to 0 or the entire period, flip the sign of the iteration step
396 // if(EPwm12Regs.CMPA.bit.CMPA == 0 || EPwm12Regs.CMPA.bit.CMPA == EPwm12Regs.TBPRD) {
397 //     pwm_iterator *= -1;
398 // }
399 // EPwm12Regs.CMPA.bit.CMPA += pwm_iterator; // sjew: increment cmpa by the iteration step
    (either +1 or -1)
400 //
401 // // sjew: if motor_counting_up not 0 increment ctrl_effort by 0.005
402 // // and if the resultant ctrl_effort exceeds the limit of 10.0 then set
    motor_counting_up to 0
403 // if(motor_counting_up) {
404 //     ctrl_effort += 0.005;
405 //     if(ctrl_effort >= 10.0) {
406 //         motor_counting_up = 0;
407 //     }
408 // // sjew: if motor_counting_up is 0, decrement ctrl_effort by 0.05
409 // // and if the resultant ctrl_effort exceeds the limit of -10.0 then
    set_motor_counting_up to 1
410 // } else {
411 //     ctrl_effort -= 0.005;
412 //     if(ctrl_effort <= -10.0) {
413 //         motor_counting_up = 1;
414 //     }
415 // }
416 // // sjew: set both motors to the current ctrl_effort
417 // setEPWM1A(ctrl_effort);
418 // setEPWM2A(ctrl_effort);
419 // sjew: store left and right encoder values
420 curr_left_enc = readEncLeft();
421 curr_right_enc = readEncRight();
422 curr_knob_enc = readEncWheel();
423 // sjew: position calculations
424 curr_p_left = curr_left_enc * 1/ left_rpf;
425 curr_p_right = curr_right_enc * 1/right_rpf;
426 // sjew: vel calculations
427 v_left = (curr_p_left - prev_p_left)/0.001;
428 v_right = (curr_p_right - prev_p_right)/0.001;
429 // u_left = curr_knob_enc; // sjew: eventually these will be determined by controls
430 // u_right = curr_knob_enc;
431
432 u_left = 0; // sjew: these are currently set to 0 to test friction comp
433 u_right = 0;
434
435 // friction comp
436 if (v_left > 0.0){
437     u_left = u_left + visc_pos* v_left + static_pos;
438 }
439 else{
440     u_left = u_left + visc_neg* v_left + static_neg;
441 }
442
443
444 if (v_right > 0.0){
445     u_right = u_right + visc_pos* v_right + static_pos;
446 }
447
448 else{
```

```

449     u_right = u_right + visc_neg* v_right + static_neg;
450 }
451 // sjew: set pwm outputs to friction compensated efforts
452 setEPWM1A(u_left);
453 setEPWM2A(u_right);
454 // sjew: store current positions for next interrupt cycle to be used as previous positions
455 prev_p_left = curr_p_left;
456 prev_p_right = curr_p_right;
457 if ((CpuTimer2.InterruptCount % 100) == 0) {
458     UARTPrint = 1;
459 }
460 }
461
462 void setEPWM1A(float controleffort) {
463     // sjew: set max and min limits for controleffort
464     if(controleffort < -10.0)
465         controleffort = -10.0;
466     if(controleffort > 10.0)
467         controleffort = 10.0;
468     int16_t offset = EPwm1Regs.TBPRD/2; // sjew: set an offset that would be a 50% duty cycle
469     float unit_step = EPwm1Regs.TBPRD/20.0; // sjew: unit step of 5% of the total period (each
1.0 in control effort corresponds to 5% of the total period)
470     EPwm1Regs.CMPA.bit.CMPA = offset+(unit_step*controleffort); // sjew: cmpa to 50% duty
cycle + (5 * controleffort)% <- controleffort ranges from -10 to 10
471 }
472 void setEPWM2A(float controleffort) {
473     // sjew: set min and max limits for controleffort
474     controleffort *= -1;
475     if(controleffort < -10.0)
476         controleffort = -10.0;
477     if(controleffort > 10.0)
478         controleffort = 10.0;
479     int16_t offset = EPwm2Regs.TBPRD/2; // sjew: set an offset that would be 50% duty cycle
480     float unit_step = EPwm2Regs.TBPRD/20.0; // sjew: unit step of 5% of the total period (each
1.0 in control effort corresponds to 5% of the total period)
481     EPwm2Regs.CMPA.bit.CMPA = offset+(unit_step*controleffort); // sjew: cmpa to 50% duty
cycle + (5 * controleffort)% <- controleffort ranges from -10 to 10
482 }
483
484 void init_eQEPs(void) {
485     // setup eQEP1 pins for input
486     EALLOW;
487     //Disable internal pull-up for the selected output pins for reduced power consumption
488     GpioCtrlRegs.GPAPUD.bit.GPIO20 = 1; // Disable pull-up on GPIO20 (EQEP1A)
489     GpioCtrlRegs.GPAPUD.bit.GPIO21 = 1; // Disable pull-up on GPIO21 (EQEP1B)
490     GpioCtrlRegs.GPAQSEL2.bit.GPIO20 = 2; // Qual every 6 samples
491     GpioCtrlRegs.GPAQSEL2.bit.GPIO21 = 2; // Qual every 6 samples
492     EDIS;
493     // This specifies which of the possible GPIO pins will be EQEP1 functional pins.
494     // Comment out other unwanted lines.
495     GPIO_SetupPinMux(20, GPIO_MUX_CPU1, 1);
496     GPIO_SetupPinMux(21, GPIO_MUX_CPU1, 1);
497     EQep1Regs.QEPCTL.bit.OPEN = 0; // make sure eqep in reset
498     EQep1Regs.QDECCTL.bit.QSRC = 0; // Quadrature count mode
499     EQep1Regs.QPOSCTL.all = 0x0; // Disable eQep Position Compare
500     EQep1Regs.QCAPCTL.all = 0x0; // Disable eQep Capture
501     EQep1Regs.QEINT.all = 0x0; // Disable all eQep interrupts

```

```

502     EQep1Regs.QPOSMAX = 0xFFFFFFFF; // use full range of the 32 bit count
503     EQep1Regs.QEPCTL.bit.FREE_SOFT = 2; // EQep unaffected by emulation suspend in Code
Composer
504     EQep1Regs.QPOSCNT = 0;
505 //     SE423 8 Lab #3
506     EQep1Regs.QEPCTL.bit.QPEN = 1; // Enable EQep
507     EALLOW;
508     // setup QEP2 pins for input
509     //Disable internal pull-up for the selected output pinsfor reduced power consumption
510     GpioCtrlRegs.GPBPUD.bit.GPIO54 = 1; // Disable pull-up on GPIO54 (EQEP2A)
511     GpioCtrlRegs.GPBPUD.bit.GPIO55 = 1; // Disable pull-up on GPIO55 (EQEP2B)
512     GpioCtrlRegs.GPBQSEL2.bit.GPIO54 = 2; // Qual every 6 samples
513     GpioCtrlRegs.GPBQSEL2.bit.GPIO55 = 2; // Qual every 6 samples
514     EDIS;
515     GPIO_SetupPinMux(54, GPIO_MUX_CPU1, 5); // set GPIO54 and eQep2A
516     GPIO_SetupPinMux(55, GPIO_MUX_CPU1, 5); // set GPIO55 and eQep2B
517     EQep2Regs.QEPCTL.bit.QPEN = 0; // make sure qep reset
518     EQep2Regs.QDECCTL.bit.QSRC = 0; // Quadrature count mode
519     EQep2Regs.QPOSCNT.all = 0x0; // Disable eQep Position Compare
520     EQep2Regs.QCAPCTL.all = 0x0; // Disable eQep Capture
521     EQep2Regs.QEINT.all = 0x0; // Disable all eQep interrupts
522     EQep2Regs.QPOSMAX = 0xFFFFFFFF; // use full range of the 32 bit count.
523     EQep2Regs.QEPCTL.bit.FREE_SOFT = 2; // EQep unaffected by emulation suspend
524     EQep2Regs.QPOSCNT = 0;
525     EQep2Regs.QEPCTL.bit.QPEN = 1; // Enable EQep
526     EALLOW;
527     // setup QEP3 pins for input
528     //Disable internal pull-up for the selected output pins for reduced power consumption
529     GpioCtrlRegs.GPAPUD.bit.GPIO6 = 1; // Disable pull-up on GPIO54 (EQEP3A)
530     GpioCtrlRegs.GPAPUD.bit.GPIO7 = 1; // Disable pull-up on GPIO55 (EQEP3B)
531     GpioCtrlRegs.GPAQSEL1.bit.GPIO6 = 2; // Qual every 6 samples
532     GpioCtrlRegs.GPAQSEL1.bit.GPIO7 = 2; // Qual every 6 samples
533     EDIS;
534     GPIO_SetupPinMux(6, GPIO_MUX_CPU1, 5); // set GPIO6 and eQep2A
535     GPIO_SetupPinMux(7, GPIO_MUX_CPU1, 5); // set GPIO7 and eQep2B
536     EQep3Regs.QEPCTL.bit.QPEN = 0; // make sure qep reset
537     EQep3Regs.QDECCTL.bit.QSRC = 0; // Quadrature count mode
538     EQep3Regs.QPOSCNT.all = 0x0; // Disable eQep Position Compare
539     EQep3Regs.QCAPCTL.all = 0x0; // Disable eQep Capture
540     EQep3Regs.QEINT.all = 0x0; // Disable all eQep interrupts
541     EQep3Regs.QPOSMAX = 0xFFFFFFFF; // use full range of the 32 bit count.
542     EQep3Regs.QEPCTL.bit.FREE_SOFT = 2; // EQep unaffected by emulation suspend
543     EQep3Regs.QPOSCNT = 0;
544     EQep3Regs.QEPCTL.bit.QPEN = 1; // Enable EQep
545 }
546
547 float readEncLeft(void) {
548     int32_t raw = 0;
549     uint32_t QEP_maxvalue = 0xFFFFFFFFU; //4294967295U
550     raw = EQep1Regs.QPOSCNT;
551     // SE423 9 Lab #3
552     if (raw >= QEP_maxvalue/2) raw -= QEP_maxvalue; // I don't think this is needed and never
true
553     return -(raw*(1/2000.0*1/20.0*TWOPI));
554 }
555
556 float readEncRight(void) {

```

```
557     int32_t raw = 0;
558     uint32_t QEP_maxvalue = 0xFFFFFFFFU; //4294967295U -1 32bit signed int
559     raw = EQep2Regs.QPOSCNT;
560     if (raw >= QEP_maxvalue/2) raw -= QEP_maxvalue; // I don't think this is needed and never
    true
561     return (raw*(1/2000.0*1/20.0*TWOPI)); // sjew: scaling factor of rotations/counts * wheel
    revs/rotation * radians/revolution
562 }
563
564 float readEncWheel(void) {
565     int32_t raw = 0;
566     uint32_t QEP_maxvalue = 0xFFFFFFFFU; //4294967295U -1 32bit signed int
567     raw = EQep3Regs.QPOSCNT;
568     if (raw >= QEP_maxvalue/2) raw -= QEP_maxvalue; // I don't think this is needed and never
    true
569     return (raw*(2*PI/4000.0)); // sjew: scaling factor of rotations/counts * wheel
    revs/rotation * radians/revolution
570 }
571
572
```